UNINASSAU

Plano de Desenvolvimento de Sistema de Consultas em Postos de Saúde

Disciplina: Arquitetura de software

Professor: Clóves Rocha

GABRIELLA GUEDES DE ALMEIDA ALVES – **01847354**

HELENA SILVA NUNES **– 01792836**

JUAN GABRIEL DIAS ARRUDA **– 01797727**

MARCUS V SOUZA DE PAULO **– 01846641**

WILLIANS EDUARDO DOS SANTOS **– 01814383**

JUAN GABRIEL DIAS ARRUDA **– 01797727**

Olinda

Março, 2026

Sistema de Marcação de Consultas em Postos de Saúde

O sistema tem como objetivo resolver um problema muito presente na sociedade: as filas para marcação de consultas nos postos de saúde do SUS. Em muitos casos, as pessoas precisam chegar muito cedo aos postos e ainda enfrentam longas esperas, o que dificulta o acesso aos serviços básicos de saúde.

De acordo com uma pesquisa divulgada pela CNN Brasil, baseada em dados do Instituto Locomotiva, as filas no Sistema Único de Saúde (SUS) representam um risco à vida da população, afetando milhões de brasileiros que dependem do serviço público.

O principal problema está no longo tempo de espera para consultas e exames, especialmente no atendimento com especialistas. Essa demora pode causar agravamento no estado de saúde dos pacientes, já que muitos relatam piora no quadro clínico enquanto aguardam atendimento.

Além disso, a lentidão no sistema faz com que parte da população busque atendimento na rede privada ou até desista de utilizar o SUS.
Diante desse cenário, surge a necessidade de uma solução que torne o processo de agendamento mais organizado, rápido e acessível, contribuindo para a redução das filas e para a melhoria do atendimento à população.

Assim, propõe-se o desenvolvimento de um sistema simples de marcação de consultas, no qual os usuários possam agendar atendimentos de forma digital, enquanto os postos de saúde tenham melhor controle dos horários e da demanda, alinhando-se ao conceito de SmartCity ao utilizar a tecnologia para melhorar serviços públicos essenciais.

Arquitetura do sistema

O sistema foi desenvolvido para permitir o agendamento online de consultas, com controle de vagas por dia, lista de espera automática e organização digital dos atendimentos, tornando o processo mais rápido, justo e acessível.

A arquitetura do sistema utiliza o padrão MVC (Model-View-Controller) em conjunto com Blueprints do Flask. O Model é responsável pelos dados e regras de negócio, como usuários, consultas e disponibilidade de vagas. O Controller recebe as requisições, processa a lógica do sistema e controla o fluxo entre os dados e a interface. A View é responsável pela apresentação das informações ao usuário. Os Blueprints são utilizados para organizar o sistema em módulos e separar funcionalidades, contribuindo para melhor manutenção, organização e escalabilidade.

A estrutura do projeto segue boas práticas, com organização por funcionalidades, separação de camadas, área de modelos de dados e preparação para testes, facilitando o desenvolvimento e o trabalho em equipe. O sistema contempla funcionalidades como cadastro de usuários, agendamento de consultas, visualização de datas disponíveis, controle automático de vagas e inclusão em lista de espera quando não houver disponibilidade.

Durante o desenvolvimento, foram considerados desafios como o grande número de usuários simultâneos, o controle preciso das vagas, a prevenção de conflitos de agendamento e a necessidade de manter o sistema simples e acessível. Além disso, são aplicadas boas práticas de segurança, como separação de responsabilidades, organização por camadas e preparação para autenticação e criptografia de senhas.

Como resultado, o sistema contribui para a redução de filas presenciais, melhora a organização dos atendimentos e amplia o acesso da população aos serviços, sendo uma solução organizada, escalável e alinhada a um cenário real de desenvolvimento.

Componentes Conectores

Com base na arquitetura MVC e na organização por Blueprints no Flask, o sistema foi estruturado em componentes independentes, cada um com uma responsabilidade específica. O componente de autenticação é responsável pelo cadastro e identificação do usuário, garantindo acesso seguro ao sistema. O componente de agendamento atua como núcleo da aplicação, sendo responsável pela criação, cancelamento e remarcação de consultas, além de validar as regras de negócio.

O componente de controle de vagas gerencia a quantidade de atendimentos disponíveis por dia, evitando conflitos de agendamento e mantendo a disponibilidade atualizada. O componente de lista de espera organiza os usuários quando não há vagas, realizando a inserção automática na fila e a realocação conforme surgem novas vagas. O sistema também conta com um componente de notificações, responsável pelo envio de confirmações e avisos aos usuários, e um componente de interface, que permite a interação e visualização das informações.

A comunicação entre os componentes ocorre por meio de conectores síncronos e assíncronos. A comunicação síncrona, via HTTP/REST, é utilizada em operações que exigem resposta imediata, como login, consulta de vagas e agendamentos. Já a comunicação assíncrona é aplicada em processos como envio de notificações e gerenciamento da lista de espera, reduzindo a sobrecarga do sistema. Além disso, o sistema adota uma abordagem orientada a eventos, reagindo automaticamente a ações como cancelamentos, novas consultas e liberação de vagas.

Padrão Arquitetural

O sistema adota como padrão arquitetural a combinação do modelo cliente-servidor com o padrão MVC (Model-View-Controller), complementado pela modularização por meio de Blueprints no Flask. Essa escolha permite estruturar a aplicação de forma organizada e escalável, alinhada às boas práticas de desenvolvimento no ecossistema Python.

No modelo cliente-servidor, há a separação entre a camada responsável pela interação com o usuário e a camada responsável pelo processamento das requisições, onde o Flask atua como servidor, gerenciando as solicitações HTTP e aplicando as regras do sistema. Dentro desse contexto, o padrão MVC organiza a aplicação em três camadas bem definidas: o Model, que centraliza os dados e regras de negócio, garantindo consistência nas operações; o Controller, que intermedia as requisições e controla o fluxo da aplicação, especialmente em cenários com múltiplos acessos simultâneos; e a View, responsável pela apresentação das informações, mantendo a interface desacoplada da lógica do sistema.

A utilização de Blueprints complementa essa arquitetura ao permitir a divisão da aplicação em módulos funcionais independentes, facilitando a organização do código, a reutilização de componentes e a manutenção do sistema. Além disso, essa abordagem contribui para reduzir o acoplamento entre as partes da aplicação, melhorar a colaboração entre desenvolvedores e tornar o sistema mais preparado para futuras expansões.

Dessa forma, o padrão arquitetural adotado se destaca por equilibrar simplicidade e eficiência, sendo adequado ao contexto do projeto ao garantir uma estrutura clara, organizada e de fácil evolução, sem a complexidade de arquiteturas mais avançadas.

Início do Desenvolvimento do Projeto (Python e Flask)

https://github.com/EduardoFilho777/Programas/blob/main/projeto.py